MVSR Engineering College · Department of CSE-Allied

FacultyLens

FACULTY PROFILE GENERATION AND STUDENT FEEDBACK EVALUATION BASED ON TEXT SUMMARIZATION AND SENTIMENT ANALYSIS

Project team

Ch. Varshith Vardhan	2451-24-751-010
D. Akhil Reddy	2451-24-751-014
M. Rohnith Reddy	2451-24-751-041

Guided by

Dr. M V R Jyothisree

Associate Professor, Dept. of CSE-Allied

Mini Project (U21PW581CT) · B.E. 3rd year, Semester 5

Academic Year 2026-2027

Project overview

FacultyLens: one page per teacher, built from facts and from what students actually wrote

Colleges collect written feedback every semester but rarely use it. FacultyLens reads every comment with AI, hides abuse and personal details, scores 8 parts of teaching with the students' own sentences as proof, and shows this next to an automatically written profile of the teacher.

Main objectives

- **O1** Write each teacher's profile from entered and imported facts (ORCID, Semantic Scholar).
- **O2** Tell if feedback is positive, negative or neutral, for each comment and for 8 parts of teaching, with proof.
- **O3** Hide abusive comments and remove names, phone numbers and roll numbers before any teacher reads them.
- **O4** Flag sarcasm, show the mix of emotions and check for gender bias across a department.
- **O5** Create the department's Feedback Analysis and Action Taken Report as PDF and Word.
- **O6** Measure accuracy against simpler methods on public student reviews.

Expected outcome

Teachers get clear, specific and safe feedback instead of just an average score, and the department gets a ready report it can act on.

Problem statement, scope and proposed solution

Problem statement: every semester, colleges collect student feedback, but the written comments are hardly read, are unsafe to share as they are, and never turn into clear steps a teacher can act on.

The problem today	How FacultyLens solves it
1 Only the 1 to 5 average is looked at. The comments that explain why are ignored.	AI reads every comment and scores 8 parts of teaching, showing the students' own sentences as proof.
2 Abusive words, names and phone numbers reach the teacher.	Abuse is hidden and personal details are removed before any teacher sees a comment.
3 Students fear being identified, so they hold back.	Students stay anonymous, and comments appear only after 5 or more students answer.
4 Teacher profiles are spread across places and out of date.	One page per teacher: an automatically written profile right next to their feedback.
5 The department feedback report is written by hand and takes days.	The report is drafted from the data; the HoD or admin approves it and downloads it as PDF or Word.

In scope: what we built

- Teacher profile page
- Score for each teaching area, with proof
- Hiding abuse and personal details
- Bias and sarcasm checks
- Feedback report (PDF, Word)
- Accuracy testing

Out of scope: not in this project

- Mixed-language comments (Telugu-English, Hinglish)
- Trend alerts
- Chatbot
- Spreadsheet upload
- Mobile app
- Training on a graphics card

Background study: existing systems and research gap

Sindhu et al. [2]

Aspect-based opinion mining on student feedback for faculty teaching evaluation, IEEE Access, 2019

What it does: A deep learning (LSTM) model finds which part of teaching a student comment is about, and whether it is positive or negative.

Limitations

- Does not hide abuse or remove personal details before teachers read comments.
- Results are not joined with a teacher profile.
- No department action report for a college.

Kastrati et al. [3]

Weakly supervised aspect-based sentiment analysis on students' reviews of MOOCs, IEEE Access, 2020

What it does: Scores aspects of online course reviews while needing only a small number of hand-labelled examples.

Limitations

- Built for online course reviews, not classroom feedback forms.
- No anonymity, moderation or approval steps for a college.
- No profile and no report a department can act on.

RateMyProfessors [5]

Public rating website; its reviews are released as a research dataset (He, Mendeley Data, 2020)

What it does: Students post star ratings and written comments about teachers on a public website.

Limitations

- Comments are published as written, with only an overall score.
- No proof per part of teaching and no sarcasm or bias checks.
- Not connected to the college, so nothing turns into action.

Research gap addressed by FacultyLens

No existing system combines all of these in one tool that runs on an ordinary college computer:

Teacher profile and feedback on one page | Score for each teaching area, with proof | Abuse and personal details removed first | Sarcasm and bias checks | Action Taken Report

Word-list sentiment (VADER [1]) is the usual simple baseline. On the RateMyProfessors reviews [5], it scored 0.741 accuracy; the model FacultyLens uses scored 0.815.

Application areas, target users and challenges

Who uses it: four roles, and each one sees only its own pages

Student

Fills two anonymous forms per course each semester: teaching (8 ratings, 3 answers) and syllabus (4 ratings, 1 comment).

Faculty

Sees safe, cleaned-up feedback; edits, imports and approves their own profile.

Head of Department

Sees the department's results and bias panel, and approves the report's actions.

Admin

Adds users, courses and semesters, runs the analysis, reviews flagged comments, downloads the report.

Application areas and use cases

- Semester feedback in engineering and degree colleges.
- Department reviews and accreditation reports that must show actions taken.
- Faculty self-improvement and appraisal discussions.
- Public teacher profile pages built from checked facts.
- Training institutes and online courses that collect written reviews.

Challenges faced by users

- Hundreds of comments per teacher; nobody has time to read them all.
- Students fear being identified, so they hold back or write abuse.
- Sarcasm ("great, another surprise test") fools tools that only count words.
- Teachers distrust a score without seeing the comments behind it.
- Gender bias in comments can go unnoticed.

Requirements and constraints

- Must run on a normal 4-core computer with 8 GB RAM; no graphics card.
- Students stay anonymous: only a one-way code per form (HMAC-SHA256 with a secret salt) stops a second answer.
- Teachers see only cleaned-up comments, and only after 5 or more answers.
- Must keep working even if the online AI service (Groq) is down.
- Easy to read, usable with a keyboard (WCAG AA), and works on phones.

Technical approach

Each comment passes through 8 steps, in the background, on a normal computer [4], [6].

1 Collect

A student gives 8 ratings and 3 answers, plus a short syllabus form. A one-way code allows one answer per form.

2 Preprocess

Clean the text, split it into sentences (NLTK) and check it is English.

3 Redact personal details

Names, phone numbers, emails, roll numbers and links are replaced with tags like [PHONE].

4 Moderate

An abuse model scores each sentence: 0.70 or more is hidden; 0.40 or more goes to the admin.

5 Sentiment and sarcasm

RoBERTa marks each sentence positive, negative or neutral; another model flags sarcasm.

6 Teaching aspects

MiniLM finds which of the 8 teaching areas a sentence is about; DeBERTa says if it is good or bad.

7 Aggregate

Scores, emotions and a bias check are added up per form and saved in MongoDB.

8 Summarise and report

Groq AI writes short summaries that point to real comments; a local model takes over if it is down.

Models and techniques (light mode)

Sentiment	`cardiffnlp/twitter-roberta-base-sentiment-latest`
Aspect matching	`sentence-transformers/all-MiniLM-L6-v2`
Aspect polarity	`yangheng/deberta-v3-base-absa-v1.1`
Toxicity	`unitary/unbiased-toxic-roberta`
Emotion	`j-hartmann/emotion-english-distilroberta-base`
Sarcasm (irony)	`cardiffnlp/twitter-roberta-base-irony`
Summary fallback	`sshleifer/distilbart-cnn-12-6`
LLM summaries	`Groq: openai/gpt-oss-120b (fallback gpt-oss-20b)`
Baselines	VADER, TF-IDF with Logistic Regression and Random Forest

How the parts work together

- The website talks to the FastAPI server, and every request carries a login token (JWT).
- When a form is submitted, the analysis runs later, so the student never waits.
- Each model loads only once, when first needed, and runs on the normal processor.
- Results are saved, so no comment is analysed twice.
- AI summaries are checked and must point to real comments, so they cannot make things up.

Feasibility and viability

Technical feasibility

- Every tool we used is well known and free: FastAPI, React, MongoDB, Hugging Face models.
- On our laptop, one response takes **310.6 ms** on average and **599.5 ms** at the slowest 5%.
- It uses at most **2.8 GB** of memory, so an 8 GB computer is enough.
- Skills: Python and React from coursework; NLP learned during the project.

Economic feasibility

Software and libraries (all open source)	₹0
AI models (free on Hugging Face)	₹0
Groq LLM (free plan; local model if it runs out)	₹0
Database (MongoDB Atlas free tier, 512 MB, or local)	₹0
Hosting (college server already owned, Slide 18)	₹0
Total to build and run the pilot	₹0

Operational feasibility

- Opens in any browser on laptops and phones; users install nothing.
- Each role sees only its own pages, so screens stay simple.
- Students finish the form in under a minute; analysis happens later.
- Fits the existing semester feedback process; no new paperwork.

Time feasibility

- Built in stages: setup, feedback forms, AI analysis, profiles, report export, then testing accuracy.
- Three members worked side by side on the website, the server and the AI part.
- All features are built and working within the semester.
- Still in progress: comments labelled by people for testing, and the teacher survey.

Viability

Used again every semester with no extra work.

Models can be upgraded later without losing old results.

Keeps working even when the online AI service is down.

Any department can use it; adding departments and courses needs no coding.

Tools and technologies used

Software requirements

Languages	Python 3.13, JavaScript (ES2022), HTML, CSS
Backend	FastAPI, Uvicorn, Pydantic v2, PyMongo Async API, PyJWT, pwdlib (bcrypt), slowapi
Frontend	React 18, Vite, React Router 7, Axios, Recharts, Tailwind CSS
NLP and ML	Hugging Face Transformers [6], PyTorch (CPU build), sentence-transformers, spaCy, NLTK, scikit-learn, Pandas, NumPy
Evaluation	rouge-score, bert-score, Cohen's kappa (scikit-learn)
Database	MongoDB (Atlas free tier or local server)
External APIs	Groq API (LLM summaries), ORCID and Semantic Scholar (publication import)
Reports	fpdf2 (PDF), python-docx (Word)
Dev tools	VS Code, Git and GitHub, Postman, pytest, Ruff, ESLint, Prettier
Operating system	Windows 11 for development; runs on Windows, Linux or macOS. Any modern browser for users.

Hardware requirements

Server or demo PC (minimum)

- 4-core 64-bit CPU; no GPU needed
- 8 GB RAM (peak use measured at 2.8 GB)
- SSD with room for Python packages and the downloaded models

Development machine used

- AMD 12-thread CPU, 15.3 GB RAM

Users

- Any laptop, desktop or phone with a browser

Network

- Internet for the first model download, Groq and Atlas; campus LAN is enough for a local install

No sensors or microcontrollers: this is a software-only system.

Input and output specifications

Inputs and sources

- **Student response:** 8 ratings (1 to 5) and 3 answers: liked, improve, other. Syllabus form: 4 ratings, 1 comment.
- **Faculty profile:** qualifications, experience, research interests, publications; typed in or imported from ORCID and Semantic Scholar.
- **Setup by admin:** departments, users, courses, enrolments and feedback cycles.
- **How it is sent:** securely over the internet (HTTPS), with a login token.

Processing

- Remove personal details and hide abusive comments.
- Mark each sentence positive, negative or neutral; spot sarcasm and emotions.
- Link sentences to the 8 teaching areas and score each area.
- Add up results per form; check for gender bias per department.
- Write summaries from real comments; draft the profile and report actions.

Outputs and formats

- **Teacher page** (web): profile next to the feedback, a chart per teaching area, proof sentences, strengths and improvements.
- **Moderation queue** for the admin.
- **Insights and bias panel** for the HoD and admin.
- **Feedback Analysis and Action Taken Report** as PDF and DOCX.
- **Accuracy results** as tables and charts.

Real sample: input and the actual output

Input: "What should the teacher improve?"

> Sir explains every concept clearly but rushes the last units. My number is 98480 12345.

Actual output from the pipeline (models run on our laptop)

> Sir explains every concept clearly but rushes the last units. My number is [PHONE].

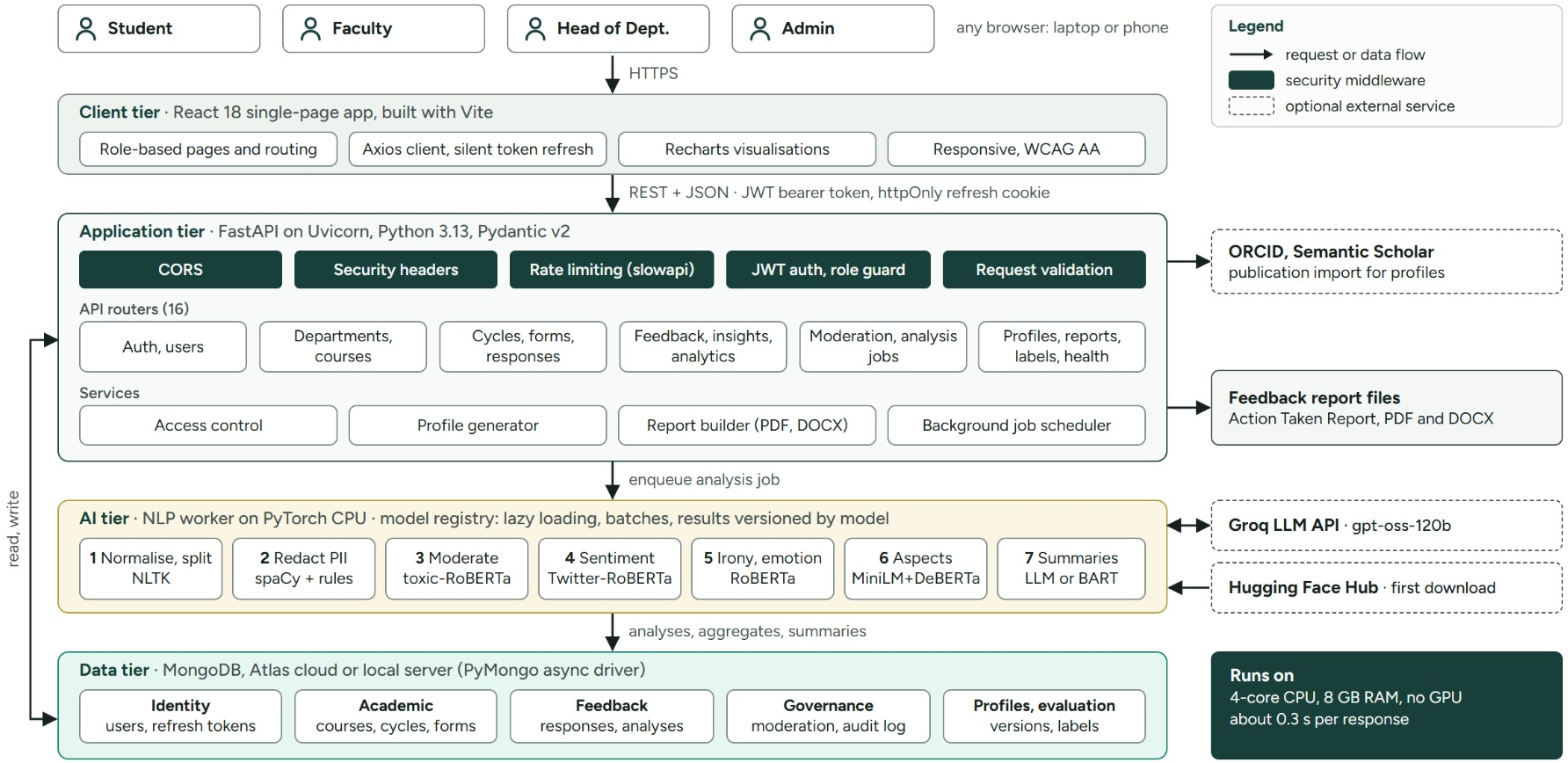
Pace & syllabus coverage: **negative** (0.45) · Overall: **neutral** (0.80) · Abuse score 0.0004: **visible** · Phone number removed

Validation and error handling

- Every form is checked; wrong input gets a clear error message.
- A second response to the same form is refused.
- Users cannot open other roles' pages; repeated wrong logins are slowed down.
- Comments stay hidden until 5 students have answered.
- If the online AI fails, a local model writes the summary.

System architecture

Four tiers: client, application, AI and data, with optional outside services.



Client tier

React website; each role gets its own pages.

Application tier

FastAPI: security checks, 16 routers, services.

AI tier

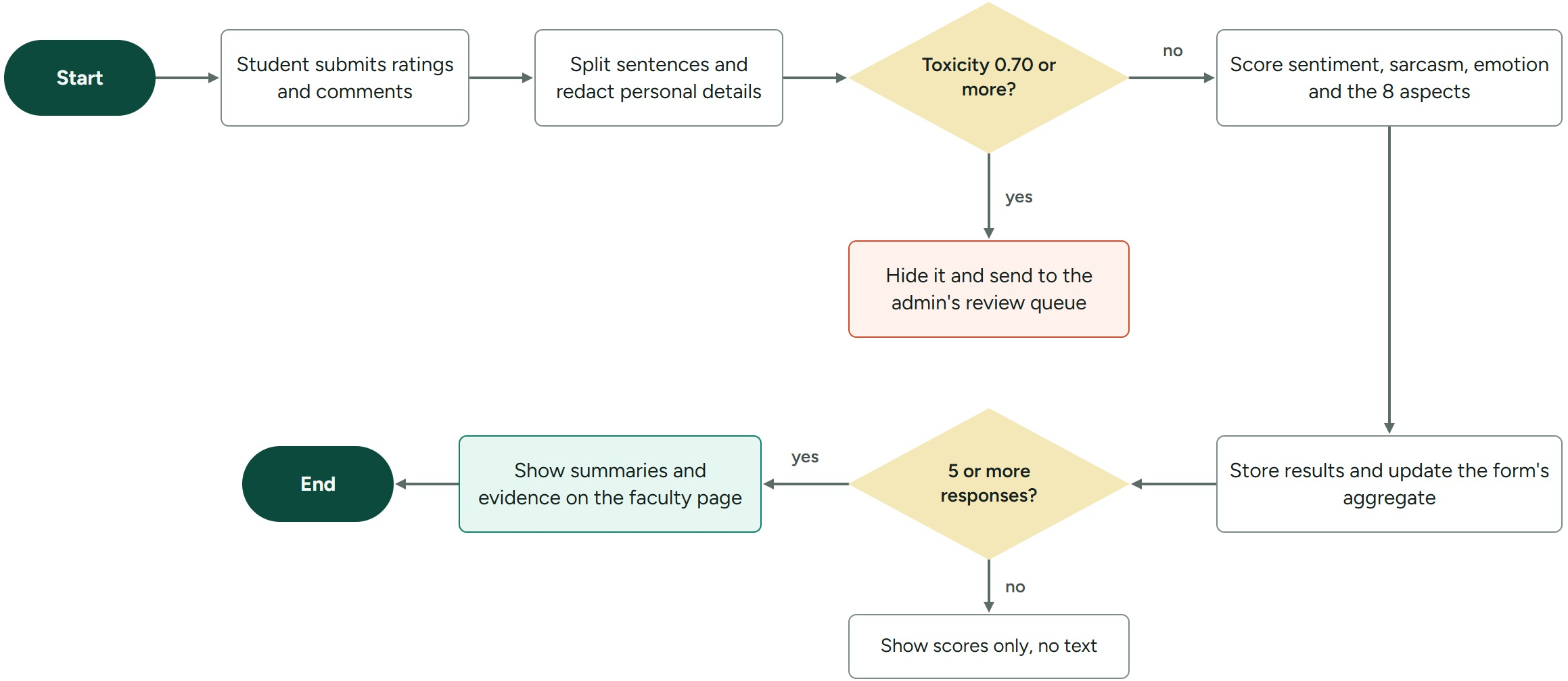
Background worker runs 7 NLP steps on the CPU.

Data tier

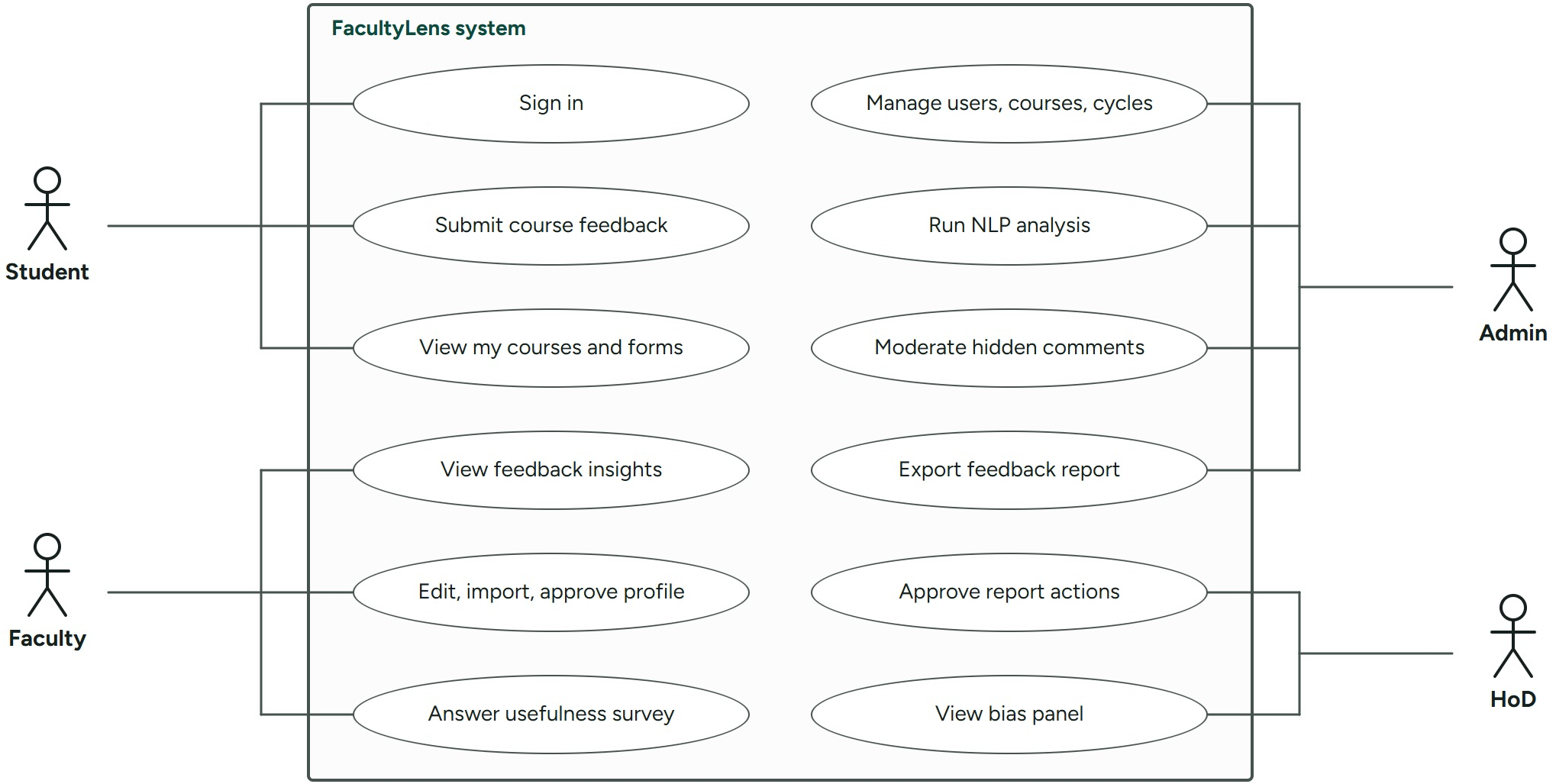
MongoDB stores results per model version.

System architecture: flowchart

Every comment goes through this path in a background job. A hidden comment is never shown until the admin decides to approve, redact or keep it hidden.



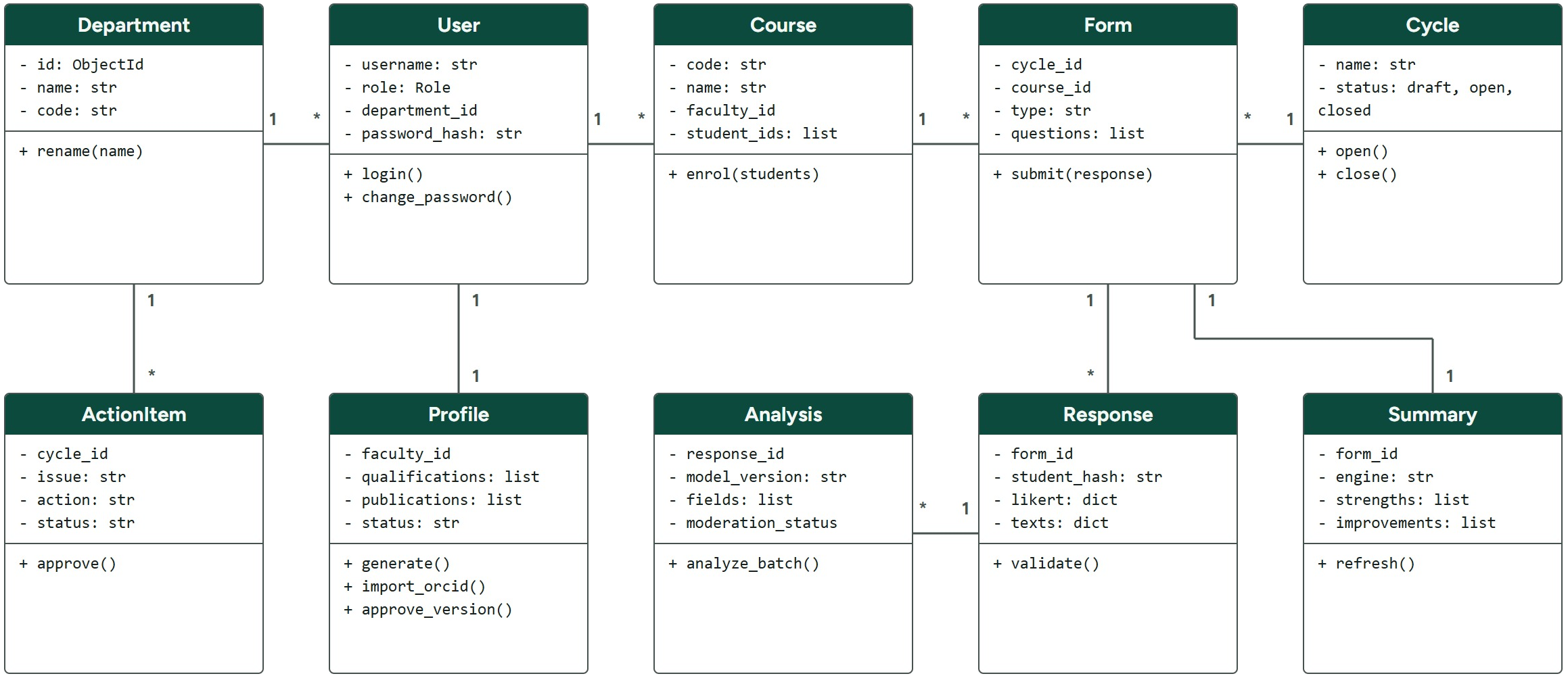
UML structure: use case diagram



Every actor also uses Sign in; lines are drawn for the Student only to keep the diagram readable.

UML structure: class diagram

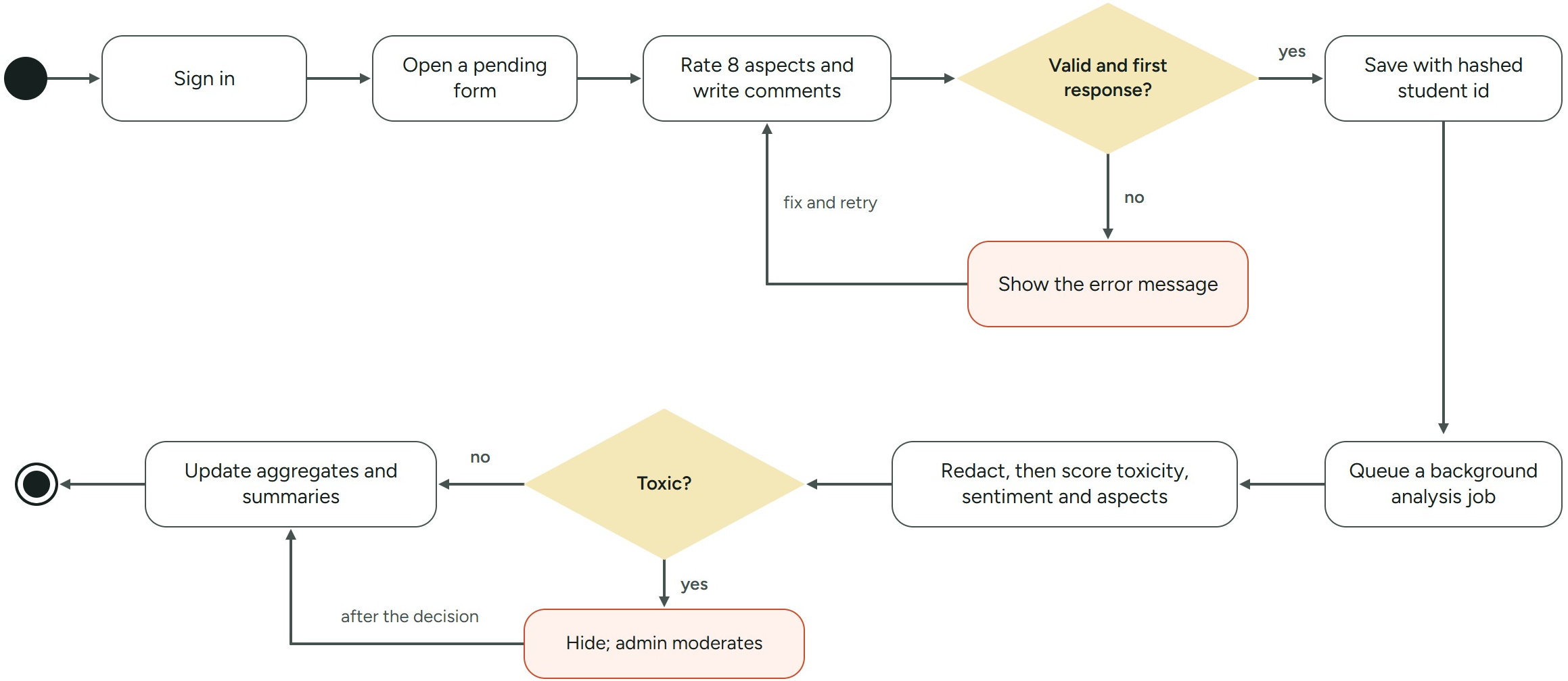
Each class is a MongoDB collection with a Pydantic schema; methods are the operations the API performs on it.



Associations with multiplicity: a department has many users and action items; a teacher teaches many courses; each form belongs to one course and one cycle, collects many responses and has one summary; each response has one analysis per model version.

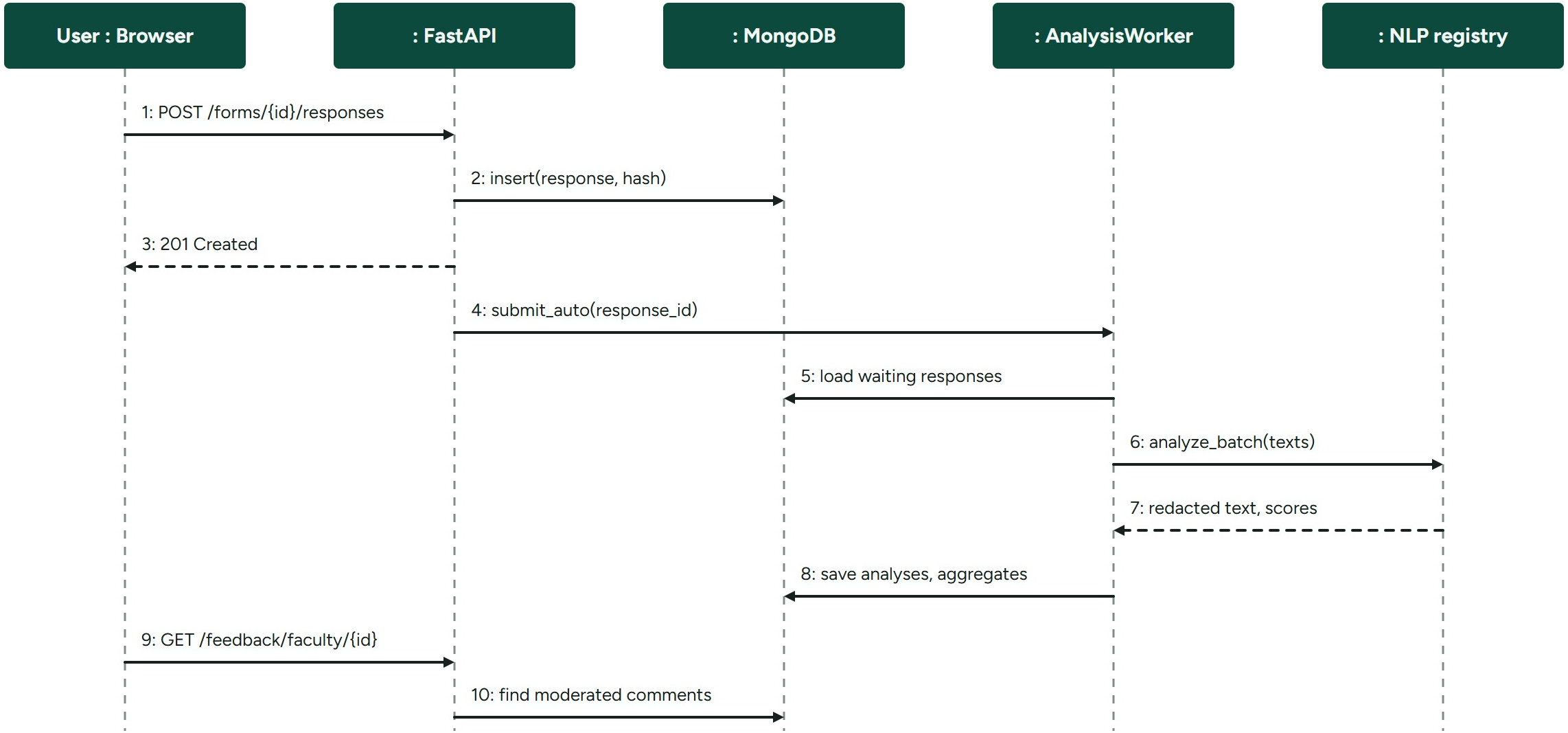
UML behaviour: activity diagram

Activity: a student gives feedback, and the system analyses it.

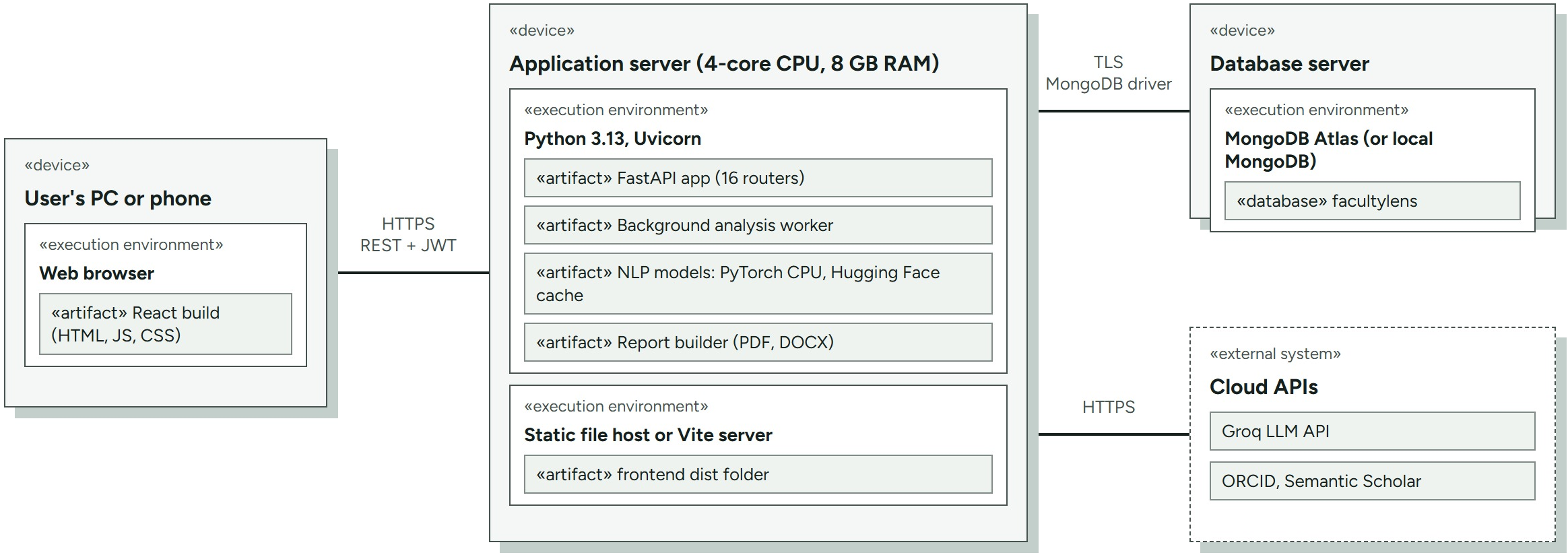


UML behaviour: sequence diagram

Submit a form (1 to 8), then view the page (9, 10).



UML deployment diagram



Development runs all three nodes on one laptop (local MongoDB). The dashed node is optional: without it, summaries fall back to local models and profiles are typed in by hand.

Implementation: environment setup

Installation steps

1. Backend environment (Python 3.13, virtual environment)

```
cd backend  ·  py -3.13 -m venv .venv  ·  .venv\Scripts\activate
```

2. CPU-only PyTorch, then the libraries

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install -r requirements.txt
python -m spacy download en_core_web_sm
```

3. Download the NLP models once

```
python -m app.nlp.registry --download
```

4. Database and demo data

```
python -m app.seed
```

5. Run the API (docs at /docs)

```
uvicorn app.main:app --reload --port 8000
```

6. Frontend

```
cd frontend  ·  npm install  ·  npm run dev
```

Configuration

- All settings live in a **.env** file that is never committed; **.env.example** lists every key.
- Key settings: database URI, JWT secret, salt for student hashes, **ML_MODE=light**, CPU thread count, Groq key and model names.
- Moderation thresholds: hide at 0.70, review at 0.40; comments shown after 5 responses.
- Models load lazily, once, never at app start.

Development tools

- VS Code, Git and GitHub for version control.
- Postman collection covering every endpoint.
- pytest, Ruff, ESLint and Prettier run in CI on every push.

Implementation: major modules

1 Authentication and roles `auth/, routers/auth.py`

Secure login with tokens, passwords stored scrambled (bcrypt), slowed-down repeated logins; each user sees only their role's pages.

2 Organisation setup `users, departments, courses`

The admin adds departments, users (many at once too) and courses, and enrols students.

3 Feedback cycles and forms `cycles.py, forms.py`

Opens a feedback round, creates the forms and accepts one anonymous answer per student per form.

4 NLP analysis pipeline `nlp/, pipeline/jobs.py`

Runs the AI steps in the background: remove personal details, check abuse, sentiment, sarcasm, emotion, teaching areas.

5 Moderation `moderation queue`

The admin decides on each flagged comment: show, clean up or keep hidden. Every decision is recorded.

6 Faculty profile generation `profiles, profile_gen.py`

Writes the profile from entered and imported facts; the teacher approves each version before it is shown.

7 Insights and bias panel `analytics.py, overview.py`

Charts per teaching area with proof, summaries, emotions, sarcasm flags and a gender-bias check.

8 Feedback report `reports.py, nlp/atr.py`

Suggests department actions from the data; approved actions download as PDF and Word.

9 Evaluation and labelling `ml/, labels.py`

Compares our models with simpler methods; two people label comments to measure accuracy.

10 Health and model status `health.py`

Shows which models are running, memory use and average speed.

How they connect: the website sends every request to the server. The server saves the data and starts the AI analysis in the background. The results are then shown on the insights, report and profile pages.

Testing and results: test cases

629 automated tests passed

12 slow tests with the real models passed

79 API endpoints, all covered in the Postman collection

0 known vulnerabilities in frontend packages

TC ID	Input	Expected	Actual	Status
TC01	Comment with phone number 98480 12345	Shown to faculty as [PHONE]	Shown as [PHONE]	Pass
TC02	Abusive comment	Hidden and sent to the moderation queue	Hidden, in the queue	Pass
TC03	Same student submits a form twice	Second response refused	Refused: “You've already submitted this form.”	Pass
TC04	Faculty opens an admin page or API	Blocked (error 403)	Blocked, 403	Pass
TC05	Form with fewer than 5 responses	Scores shown, comment text hidden	Scores only, text hidden	Pass
TC06	Analysis run twice on the same data	Saved results reused	Nothing analysed again	Pass
TC07	No Groq key, or Groq down	Summary still written, no crash	Template summary written	Pass
TC08	Issue raised for one teacher only	Not reported as a department issue	Not in department findings	Pass
TC09	Malformed teacher id	Clear error message (422)	422 with the reason	Pass
TC10	Admin approves a held comment	Scores and summary rebuilt	Rebuilt	Pass
TC11	Comment ids sent to faculty	One student's answers cannot be linked	Not linkable	Pass

Testing and results: accuracy and limitations

Sentiment benchmark (611 test comments)

Public RateMyProfessor reviews [5]; the right answer comes from each student's star rating.

Model	Accuracy	Macro-F1	F1 neutral
VADER (lexicon baseline)	0.741	0.517	0.041
TF-IDF + Random Forest	0.764	0.552	0.068
Twitter-RoBERTa (zero-shot)	0.815	0.647	0.228
DistilBERT (fine-tuned on CPU)	0.807	0.653	0.263
TF-IDF + Logistic Regression	0.789	0.671	0.383

We use Twitter-RoBERTa (green bar): most accurate, and needs no training.

Moderation and redaction

Tested on 50 sentences we wrote by hand.

Check	Precision	Recall	F1
Abuse hidden	0.733	1.000	0.846
Any personal detail	1.000	1.000	1.000
Student names	1.000	0.750	0.857
Phone numbers	1.000	1.000	1.000
Roll numbers	1.000	1.000	1.000

Performance on a laptop CPU (553 responses)

310.6 ms
average per response

599.5 ms
95th percentile

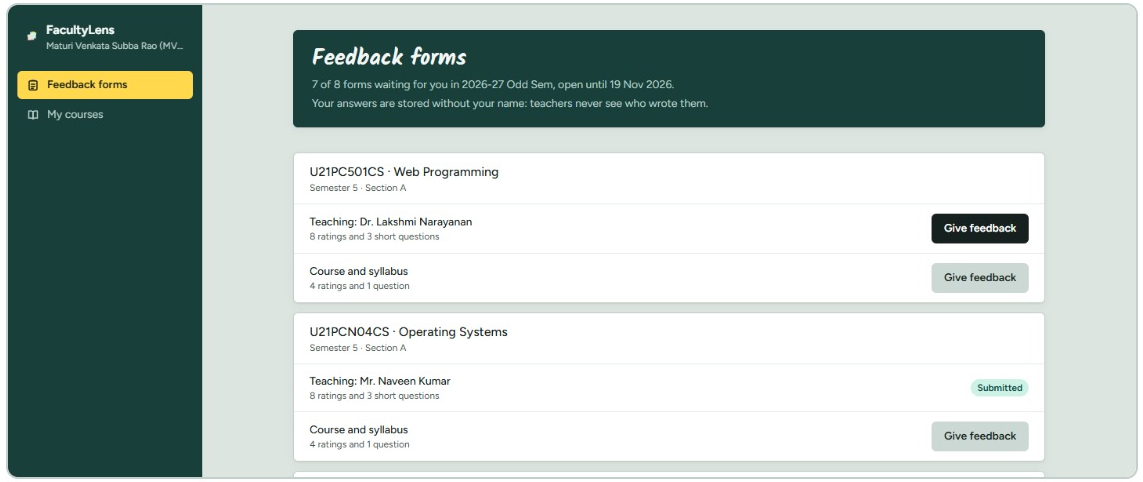
2.8 GB
peak RAM, no GPU

A class of 60 students is analysed in well under a minute. Every number here comes from our own test runs.

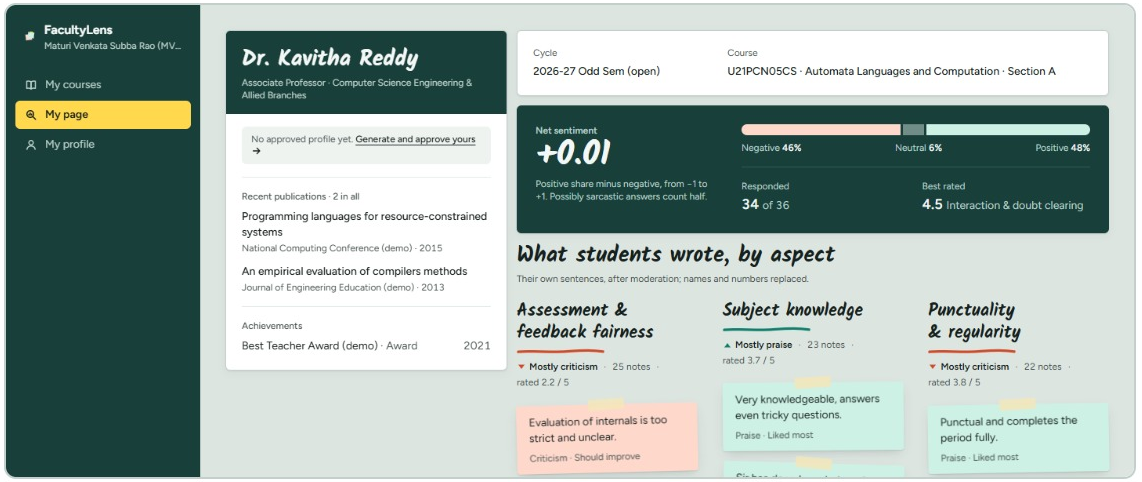
Observed limitations and their mitigation

- Neutral comments are hard for every model (score at most 0.383), so unclear ones are marked as unsure.
- 4 harmless sentences were hidden by mistake, but a person reviews every hidden comment.
- Star ratings are only a rough guide; a set labelled by people is in progress.
- 1 in 4 student names was missed; improving this is future work.

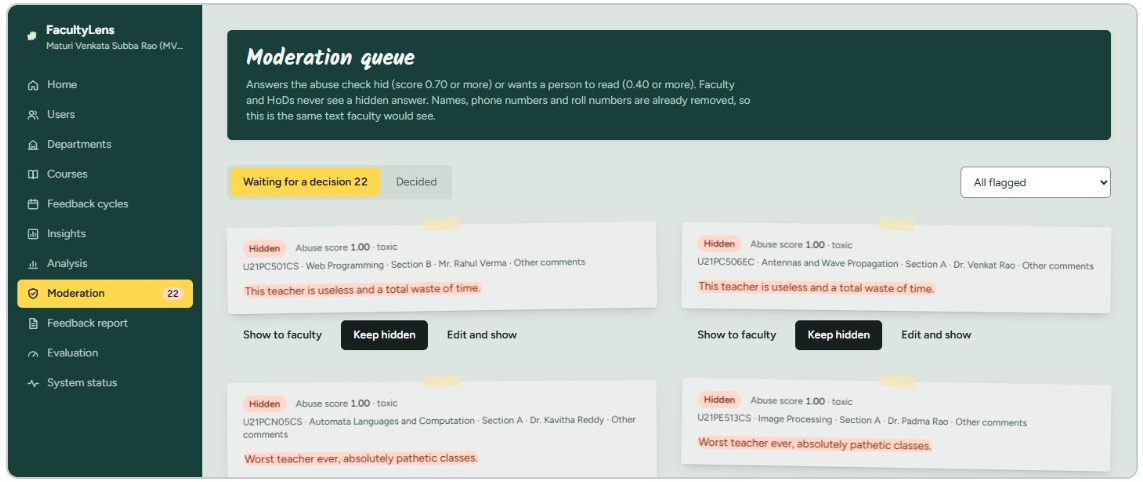
Testing and results: screenshots



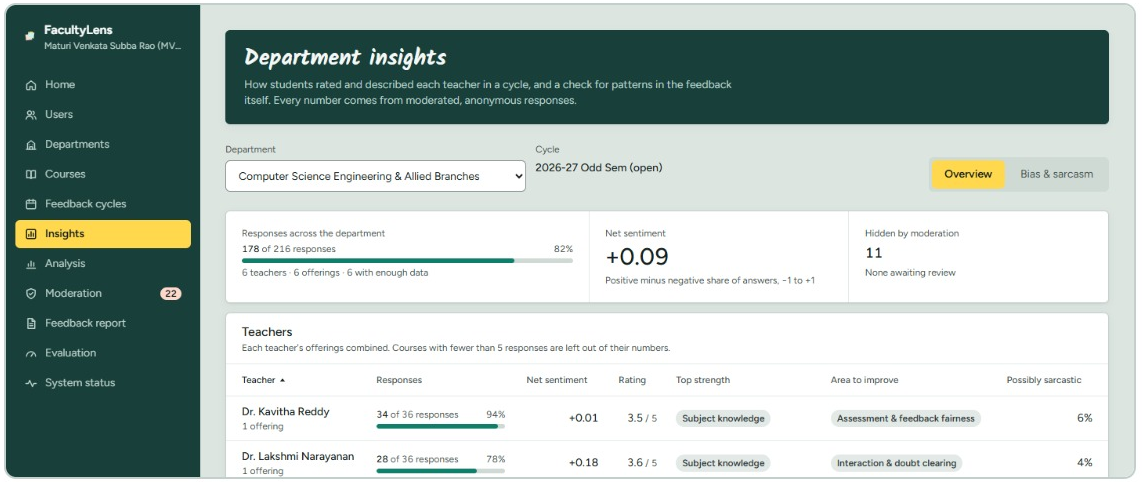
Student: feedback forms, answered anonymously



Faculty: profile beside sentiment and notes sorted by aspect



Admin: moderation queue with abuse scores



Admin and HoD: department insights

Certification details

No certification has been completed yet. Each member's planned free course is listed; certificates will be added once completed.

Ch. Varshith Vardhan

Certification
Not completed

Issued by
Planned: freeCodeCamp, Front End Development Libraries

Status
Not completed

Skills and relevance to FacultyLens
React, used to build the FacultyLens website

Certificate image: not completed yet

D. Akhil Reddy

Certification
Not completed

Issued by
Planned: MongoDB University, MongoDB Skill Badges

Status
Not completed

Skills and relevance to FacultyLens
Database design and queries, used for the MongoDB collections

Certificate image: not completed yet

M. Rohnith Reddy

Certification
Not completed

Issued by
Planned: Kaggle Learn, Intro to Machine Learning

Status
Not completed

Skills and relevance to FacultyLens
Machine learning basics, used for the baseline models

Certificate image: not completed yet

Impact and benefits

Benefits to each stakeholder

- **Teachers:** know exactly which aspect to improve, with the students' own words as proof, and never see abuse.
- **Students:** stay anonymous and see that feedback leads to action.
- **HoDs:** see department patterns and approve actions in one place.
- **Administration:** a publishable department report, with evidence behind every action.

Savings, efficiency and wider impact

- **Time:** each response is analysed in about 0.3 s on a CPU, instead of a person reading every comment.
- **Cost:** free models on computers the college already has; no graphics card or paid licence.
- **Accuracy:** a result for each teaching area, not one average; every comment with personal details in our test was caught.
- **Educational impact:** better teaching through specific, fair feedback.
- **What to measure:** how many students answer, how scores change each semester, and what teachers say in a survey.

Advantage over the systems on Slide 4	Sindhu et al. [2]	Kastrati et al. [3]	RateMyProfessors [5]	FacultyLens
Score for each teaching area	**Yes**	**Yes, for online courses**	No, overall score only	**Yes, with the students' sentences as proof**
Abuse and personal details removed	No	No	Partly, by manual reports	**Yes, before anyone sees them**
Teacher profile on the same page	No	No	No	**Yes, written automatically**
Sarcasm and bias checks	No	No	No	**Yes**
Action taken report	No	No	No	**Drafted from the data, PDF and Word**

Business model and deployment strategy

Users and value

- **Target users:** engineering and degree colleges: their students, teachers, HoDs and admins.
- **Value:** safe, specific feedback with proof, plus a ready department report.
- Runs on computers the college already has; no graphics card.

Proposed revenue model

- **Today:** no revenue. Free and open source for any college to host itself.
- **Future option:** a yearly hosted plan per institution.
- **Paid add-ons:** setup, staff training and support.

Deployment method (Slide 13)

- One college server on the campus network (4-core CPU, 8 GB RAM) runs FastAPI, the analysis worker and the React build.
- MongoDB Atlas free tier or a local MongoDB server.
- Optional over HTTPS: Groq, ORCID, Semantic Scholar. Users only need a browser.

Promotion and adoption

- **1. Pilot** in the Dept. of CSE-Allied, MVSR, for one feedback cycle.
- **2. Demonstrate** to faculty and HoDs; collect the usefulness survey.
- **3. Extend** to all departments of the college.
- **4. Share** with other colleges through faculty networks and GitHub.

Maintenance and updates

- Back up the database before every feedback cycle.
- Upgrade models safely: results are stored per model version, so old results stay.
- 629 automated tests and CI run on every change before it is used.
- Setup and admin steps are documented in the README.

Conclusion and future enhancements

Conclusion

Colleges collected student comments but rarely used them, and sharing them as they were was unsafe. FacultyLens reads every comment on a normal computer, removes abuse and personal details, scores 8 parts of teaching with proof, and shows this next to an automatically written profile. On public reviews it was right 81.5% of the time, and it takes about 0.3 seconds per response.

Objectives from Slide 2

✓	**O1** Unified faculty profile page	Achieved
✓	**O2** Aspect sentiment with highlighted evidence	Achieved
✓	**O3** Moderation and personal-detail redaction	Achieved
✓	**O4** Bias and sarcasm panel	Achieved
✓	**O5** Feedback report export (PDF, DOCX)	Achieved
🕓	**O6** Accuracy testing: done on public data; people's labels and teacher survey in progress	Partly

Future enhancements

- **Features:** trends across semesters with alerts; a mobile app for students.
- **Accuracy:** train on real MVSR comments; catch more names; ignore meaningless answers.
- **Languages:** understand Telugu-English and Hindi-English comments.
- **Connect:** one login with the college portal, and import students from it.
- **Grow:** handle many colleges at once.
- **Safety:** activity logs, automatic backups, regular password changes.
- **Deployment:** host on a college server or cloud with HTTPS and a custom domain.

Research references

No.	Reference (IEEE format)	Type	Cited on slide
[1]	C. J. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in *Proc. Int. AAAI Conf. Web and Social Media (ICWSM)*, 2014.	Research paper	4
[2]	I. Sindhu, S. M. Daudpota, K. Badar, M. Bakhtyar, J. Baber and M. Nurunnabi, "Aspect-based opinion mining on student's feedback for faculty teaching performance evaluation," *IEEE Access*, vol. 7, 2019.	Research paper	4, 17
[3]	Z. Kastrati, A. S. Imran and A. Kurti, "Weakly supervised framework for aspect-based sentiment analysis on students' reviews of MOOCs," *IEEE Access*, vol. 8, 2020.	Research paper	4, 17
[4]	D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. draft, 2026. [Online]. Available: web.stanford.edu/~jurafsky/slp3	Book	6
[5]	J. He, "Big data set from RateMyProfessor.com for professors' teaching evaluation," Mendeley Data, V2, 2020, doi: 10.17632/fvtfjyvw7d.2.	Dataset	4, 15, 17
[6]	Hugging Face, "Transformers documentation," 2026. [Online]. Available: huggingface.co/docs/transformers	Official documentation	6, 8

Every source discussed on Slide 4 ([1], [2], [3] and [5]) appears here with the same number. Pretrained models are used from their Hugging Face model pages under their own licences.

THANK YOU

Questions & Answers

FacultyLens: Faculty Profile Generation and Student Feedback Evaluation based on Text Summarization and Sentiment Analysis

Ch. Varshith Vardhan
2451-24-751-010

D. Akhil Reddy
2451-24-751-014

M. Rohnith Reddy
2451-24-751-041

Guided by **Dr. M V R Jyothisree**, Associate Professor, Dept. of CSE-Allied